FDDT — Fraud Document Detection Tool

(Formerly the "Document Authenticator Tool" / DocAuth. The product name is a display-level rename only — the repo folder, `docauth` database/containers and code identifiers are unchanged. Its single source is `APP_NAME` / `APP_FULL_NAME` in `backend/app/core/config.py` and `frontend/src/lib/appInfo.ts`.)

AI-powered document authentication, fraud detection, and case-management platform. Full documentation — architecture, every pipeline stage, database schema, API reference and deployment notes — is in `docs/`.

What it does: users submit a case of supporting PDF documents (vendor invoices, school documents, quotations, travel claims, payment evidence) — each upload is validated first (PDF only, ≤ 10 MB, not corrupted, not password-protected). A Celery pipeline OCRs each document (Azure Document Intelligence), then classifies it and extracts its fields (Azure OpenAI, behind the `LLMService` abstraction — see `app/services/llm_service.py`). It then runs field validation, issuer verification, cross-document consistency checks, and PDF forensics (metadata, Error Level Analysis, copy-move, ghost text on converted scans, font consistency, spelling (English), duplicate detection, visual-inconsistency review, signature/stamp detection). A transparent, weighted rules engine turns the results into an explainable risk score, and the case goes to a Reviewer L1 who approves, rejects or escalates it; an escalated case moves to the Reviewer L2 tier, which resolves it. Every step is written to an append-only audit log, and reviewers can export a per-case forensic report as PDF.

Stack: FastAPI + PostgreSQL (Row-Level Security, PgBouncer) + Celery/Redis (three queues, fair-share, global Azure rate limits) backend; React + TypeScript (Vite) frontend with login, case queue, case detail (live processing status, overlays, risk reasons, audit timeline), new-case/upload, company settings and platform-admin screens. Multi-tenant: several client companies share one deployment, each seeing only its own data.

User Manuals

- 🏢 **Company User Manual** — Step-by-step operational guide for client organizations covering all 3 company roles (**User/Submitter**, **Reviewer L1/Analyst**, **Reviewer L2/Supervisor**), case submission, bulk uploads, review workflows, automated checks, and company settings.
- ⚙️ **Platform Administrator Manual** — Comprehensive guide for platform operators covering tenant provisioning, company storage/file limits, user account management across all 4 roles, password resets, global rule templates, queue monitoring, usage tracking, and audit governance.

Prerequisites

- Docker Desktop (for Postgres, Redis and PgBouncer)
- Python 3.11+ (developed/tested against 3.14)
- Node.js 20+ (developed/tested against 24)

Setup

```
# 1. Start Postgres + Redis + PgBouncer
docker compose up -d

# 2. Backend: create venv, install deps, configure env
cd backend
python -m venv .venv
./.venv/Scripts/activate        # Windows
# source .venv/bin/activate     # macOS/Linux
pip install -r requirements.txt
cp .env.example .env            # edit if you changed any docker-compose ports/creds

# 3. Apply migrations
alembic upgrade head

# 4. Seed the platform admin (admin@example.com / ChangeMe123!)
python seed.py

# 5. Run the API
uvicorn app.main:app --reload
```

API docs: http://localhost:8000/docs

```
# 6. Celery workers — one per processing queue, each in its own terminal
#    (scale each independently; see docs/processing-queues.md)
cd backend
./.venv/Scripts/activate   # or the equivalent for your shell
celery -A app.tasks.celery_app worker -Q extraction_queue -n extraction@%h --pool=threads -c 4 --
loglevel=info
celery -A app.tasks.celery_app worker -Q vision_queue     -n vision@%h     --pool=threads -c 4 --
loglevel=info
celery -A app.tasks.celery_app worker -Q forensics_queue,celery -n forensics@%h --pool=threads -c 4 --
loglevel=info
# optional: the scheduler + the worker for its jobs (queue metrics every minute,
# stuck-document recovery every 5 minutes, nightly usage reconciliation)
celery -A app.tasks.celery_app beat --loglevel=info
celery -A app.tasks.celery_app worker -Q housekeeping_queue -n housekeeping@%h --pool=solo --loglevel=info
```

On Linux/macOS use `./start-worker.sh extraction|vision|forensics` and `./start-beat.sh` (beat and the housekeeping worker in one process — `--beat` does not work on Windows; pool sizes from `*_WORKER_CONCURRENCY`; forensics uses prefork, one process per core). On Windows use `--pool=threads` (prefork doesn't work there). Or run them in Docker: `docker compose --profile workers up -d --scale forensics-worker=3`. Without the workers, uploaded documents sit at `processing_status: "pending"` forever — the API only enqueues the jobs.

PgBouncer (recommended): `docker compose up -d` also starts PgBouncer on :6432 (transaction pooling). Set `DATABASE_POOLER_HOST=localhost` in `backend/.env` to route the app through it (each process then keeps no pool of its own). See `docs/processing-queues.md` for the connection arithmetic, fair-share scheduling and the calibrated Azure rate limits (`python -m scripts.calibrate_llm_tokens` re-measures real token usage).

Multi-tenancy: `admin@example.com` is a *platform admin* (no company). Create a company under **Platform › Companies** and its users under **Platform › Users**, then sign in as one of them to submit cases. The running app connects to Postgres as the RLS-enforced `fddt_app` role created by the migration (`DATABASE_APP_*` in `.env`). See `docs/multi-tenancy.md`.

```
# 7. In a third terminal: frontend
cd frontend
npm install
npm run dev
```

Open http://localhost:5173 and sign in with `admin@example.com` / `ChangeMe123!`. See `frontend/README.md` for details, and `docs/deployment/production-deployment.md` for the production plan on Azure (architecture, service tiers, costs, runbook). The demo is live on Azure: https://docauth-web.icysand-160befae.eastus.azurecontainerapps.io/

Login: admin@example.com / tTHt0pRc1ISDF6!9

Verifying the document processing pipeline

1. Confirm `AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT` / `_KEY` and `AZURE_OPENAI_KEY` / `_ENDPOINT` / `_DEPLOYMENT_NAME` are set in `backend/.env` (see `.env.example`) and the Celery worker (step 6 above) is running.
2. In the UI (as a company user): submit a new case, upload a PDF, then open the case detail page. Each document shows a status badge (Pending → Processing… → Complete/Failed) that updates on its own every few seconds while still in flight (the page polls `GET /cases/{id}` — no manual refresh needed). Once complete, the document type, its core fields (issuer/date/amount/reference #), and every other field the model found are shown inline, each flagged `uncertain` if the model wasn't confident.
3. To inspect the raw result directly (e.g. to review extraction quality against a source document, including Arabic text) instead of via the UI:

   ```
   # Case detail via API — includes document_type/processing_status/
   # extracted_fields per document:
   curl http://localhost:8000/cases/<case_id> -H "Authorization: Bearer <token>"

   # Or straight from Postgres, including the full OCR text
   # (documents.ocr_text) that the LLM actually saw — useful for
   # checking OCR accuracy on its own, independent of the LLM step:
   docker exec docauth-postgres psql -U docauth -d docauth -c \
     "SELECT document_type, processing_status, extracted_fields, ocr_text FROM documents WHERE id =
   '<document_id>';"
   ```

4. A failed document shows its `processing_error` inline in the UI (and in `documents.processing_error` / the `audit_log` table) instead of silently staying incomplete.
5. `backend/tests/test_document_processing.py` covers the task's success/ failure/unknown-id logic against mocked OCR/LLM services (no live Azure calls, no Celery worker needed) — run it with the rest of the suite via `pytest`. It does not (and cannot) verify real-world OCR/LLM accuracy — that has to be checked against actual sample documents, per step 3 above.

Case report export

Reviewers can click **Export report** on Case Detail to generate a standalone PDF (`POST /cases/{id}/reports` ; history at `GET /cases/{id}/reports`). Sections, in order: 1 Case Details, 2 Executive Summary, 3 Explainable Findings, 4 All Checks, 5 Exceptions, 6 Limitations and Assumptions, 7 Audit Trail, 8 Appendix (fields, SHA-256 hashes, technical parameters), 9 PDF Highlighted Regions. Every highlight has a region ID (R1, R2, …) that Section 5 links to. Generation is synchronous (`app/services/case_report_*.py`). Reports are stored in Blob Storage under `companies/{company}/cases/{case}/reports/` with a row in `case_reports` ; every export adds a new report, none are overwritten. The findings drawn onto the report's page renders are separate from the live Case Detail overlays, which

never save anything, and the original files are only ever read. Run `alembic upgrade head` to create the `case_reports` table.

Highlight colours (live overlays and the report use the same convention): solid red = ELA, solid orange = copy-move, dashed blue = model judgment (visual review, signature — "approximate"), solid purple = rule-based field exception (a failed field-validation sub-check, or a cross-document mismatch, shown on **both** documents), solid fuchsia = font mismatch, solid teal = deleted / shortened text (ghost text). Every check and finding is shown short first (one line with the values), the full explanation behind it.

Field locations

Field exceptions can only be highlighted if each extracted field's position is known. At extraction time the OCR word boxes are now kept long enough to match each extracted value back onto the page, and the result is stored on the field as a normalized `bounding_box` (`app/services/field_locator_service.py`). This is text matching, so positions are approximate; a field that can't be matched simply has no highlight. Documents extracted before this existed can be backfilled (re-runs only the Azure Layout call — no LLM, no value changes):

```
cd backend
python backfill_field_locations.py CASE-XXXXXXXX      # or no args for every document
```

Tests

```
cd backend
pytest
```

Tests run against an in-memory SQLite DB, independent of the Postgres container, so `pytest` works without `docker compose up` . The Row-Level Security, PgBouncer and rate-limiter suites need the real services — see `docs/deployment/local-setup.md` .

Project layout

```
backend/    FastAPI app, Celery tasks, Alembic migrations, tests
frontend/   React + TypeScript SPA (Vite)
docs/       Architecture, pipeline, database, API and deployment docs
sample-documents/   Sample PDFs for testing
docker-compose.yml  Local PostgreSQL + Redis + PgBouncer (+ optional Linux workers)
pgbouncer/          PgBouncer config for local development
```

See `docs/README.md` for the full documentation index.